

CAPSTONE PROJECT REPORT

Selenium Python Automation Framework

Web Application Automation using Selenium WebDriver

Application Under Test

Automation Exercise

Submitted By

Sutanu Pal

B.Tech – Computer Science and Engineering (AI & ML)

Institute of Engineering and Management, Kolkata

Enrollment Number: 12023002028062

Email: sutanu.pal2023@iem.edu.in

Academic Project / Capstone Submission

2026

Declaration

I, Sutanu Pal, a student of B.Tech in Computer Science and Engineering (Artificial Intelligence and Machine Learning) at the Institute of Engineering and Management, Kolkata, hereby declare that this capstone project report titled "Selenium Python Automation Framework" represents the work carried out as part of my academic project. The implementation, test cases, framework structure, execution results, and documentation presented in this report correspond to the project developed for the specified application under test.

The project has been developed for educational and portfolio purposes. Actual test credentials used during local execution are not included in the repository or this report.

Acknowledgement

I would like to express my sincere gratitude to the Institute of Engineering and Management, Kolkata, for providing the opportunity to undertake this capstone project. I am also thankful to the faculty and mentors for their guidance and support throughout the development and documentation of the project.

I would also like to acknowledge the Automation Exercise website, which provided the application environment used for practicing and demonstrating Selenium-based web automation.

Table of Contents

Section	Page No.
1. Abstract	5
2. Introduction	6-7
3. Project Objectives	8
4. Application Under Test	8
5. Tools and Technologies	8
6. Framework Architecture	9
7. Project Structure	10
8. Framework Components	11
9. Test Scenarios	12
10. Implementation Details	12
11. Test Execution	13
12. Test Results and Outputs	13
13. Screenshots and Evidence	14
14. Challenges and Solutions	15
15. Conclusion	15
16. Future Enhancements	15
17. References	16
18. GitHub Repository	16

1. Abstract

This project presents the design and development of a reusable Selenium Python automation testing framework for testing an e-commerce web application. The main objective is to automate important user interactions and validate application functionalities in a structured, reliable, and maintainable manner. The framework is developed using Python and Selenium WebDriver, with PyTest and Python Unittest used for test execution and validation.

The framework follows the Page Object Model (POM) design pattern, which separates web-page elements and their corresponding actions from the test cases. This improves code readability, reduces duplication, and makes the framework easier to maintain and extend. Separate page classes are implemented for user login and product search, while reusable utility classes handle configuration reading, CSV test data, WebDriver creation, and screenshot capture.

Two major application scenarios were automated: user login and product search. The framework uses configuration management to store application settings such as the URL, browser, and timeout values, while test inputs are maintained separately using CSV files. It also includes failure screenshot capture for PyTest tests and uses the pytest-html plugin to generate an HTML execution report, providing useful evidence of the test execution.

The completed framework was successfully executed with three automated test cases, resulting in 3 passed, 0 failed, and 0 skipped tests. The project deliverables include the complete source code, configuration files, test data template, HTML execution report, screenshots, README documentation, and project report. Overall, the project demonstrates how Selenium Python can be used to create a structured, reusable, and maintainable automation framework that can be extended with additional test scenarios in the future.

2. Introduction

Software testing is an essential and critical part of the Software Development Life Cycle (SDLC). It helps ensure that a software application performs according to its expected requirements and provides a reliable experience to its users. Testing allows defects, functional issues, and unexpected behavior to be identified before an application is released or delivered to end users. As modern web applications become increasingly complex, testing a large number of functionalities manually can require significant time and effort.

Manual testing is useful for exploratory testing and scenarios that require human judgment, but repeatedly executing the same test cases manually can become repetitive, time-consuming, and prone to human error. For example, performing the same login or product-search operation every time a new version of an application is released requires the tester to repeat the same sequence of actions. Test automation addresses this problem by allowing predefined test scenarios to be executed automatically and repeatedly with greater speed, consistency, and accuracy.

Test automation is particularly useful for regression testing, where previously implemented functionalities need to be verified after changes are made to an application. Automated tests can be executed multiple times without requiring the tester to manually perform every step. This reduces repetitive effort and allows testers to focus more on analyzing application behavior, identifying defects, and improving overall software quality.

Selenium WebDriver is a widely used open-source framework for automating web browsers. It allows automation scripts to interact with web applications in a manner similar to a real user. Selenium can perform actions such as opening web pages, clicking buttons, entering text into input fields, navigating between pages, and validating information displayed on the screen. Selenium WebDriver supports multiple browsers and programming languages, making it a popular choice for web application automation.

In this project, Selenium WebDriver is used with Python to automate and test the Automation Exercise e-commerce web application. The application provides several functionalities that are suitable for practicing web automation, including user authentication, product browsing, product searching, and other e-commerce operations. For this project, the focus is primarily placed on the user login and product search scenarios.

Instead of implementing all automation logic in a single test script, a structured automation framework has been developed. A framework-based approach provides better organization and makes the automation project easier to maintain and expand. The framework separates different responsibilities into dedicated components, including test cases, page objects, configuration files, test data, and reusable utility classes.

The Page Object Model (POM) design pattern is used to separate web-page-specific elements and operations from the test cases. For example, the login page has its own page class containing the required locators and actions for entering credentials and performing login. Similarly, product-related operations are maintained separately in the products page class. This separation makes the test cases simpler and reduces duplication of Selenium code.

The framework also includes utility classes for performing common operations. The ConfigReader utility is responsible for reading configuration values such as the application URL, browser, and timeout settings. The CSVReader utility is used to read test data from CSV files, allowing test data to be maintained separately from the automation code. The DriverFactory is responsible for creating and configuring the Selenium WebDriver, providing a centralized mechanism for browser initialization. A ScreenshotUtility is also included to capture screenshots when PyTest test cases fail, which provides useful visual evidence for debugging.

Both PyTest and Python Unittest are incorporated into the project to demonstrate different approaches to test execution. PyTest is used as the primary testing framework and provides features such as test discovery, fixtures, assertions, and HTML reporting. Python's built-in Unittest framework is also used to implement and execute a separate login test case.

The framework further implements configuration management and data-driven testing. Configuration values are maintained in a separate config.ini file, while test inputs are maintained using CSV-based test data. This approach prevents important configuration and test data from being hard-coded directly into the test scripts. It also makes the framework more flexible, since values can be modified without making significant changes to the automation logic.

For test execution and documentation, the framework uses pytest-html to generate an HTML test report. The report provides a clear overview of the executed test cases and their results. In addition, screenshots generated during failed PyTest executions are stored separately as project evidence. These outputs help in understanding the behavior of the automation framework and provide supporting evidence for the completed test execution.

The overall goal of this project is therefore not only to automate individual test cases, but also to demonstrate the development of a maintainable, reusable, and scalable Selenium Python automation framework. By combining Selenium WebDriver, Page Object Model, PyTest, Unittest, utility classes, configuration management, CSV test data, screenshot handling, and HTML reporting, the project provides a structured foundation that can be extended with additional application functionalities and test scenarios in the future.

3. Project Objectives

- Develop a reusable Selenium Python automation framework.
- Implement the Page Object Model for maintainable page-specific automation.
- Use PyTest for test discovery, fixtures, assertions, and execution.
- Implement a separate Unittest-based login test.
- Manage application configuration using a configuration file.
- Manage test inputs using CSV data.
- Create reusable utility classes for common framework operations.
- Capture screenshots when PyTest tests fail.
- Generate an HTML execution report using pytest-html.
- Document the framework and maintain the project in GitHub.

4. Application Under Test

Application: Automation Exercise

Website: <https://automationexercise.com/>

Automation Exercise is an E-Commerce practice website containing functionality such as user registration, login, product browsing, product searching, and other shopping-related operations. For this capstone, the Login and Product Search flows were selected.

5. Tools and Technologies

Technology / Tool	Purpose
Python	Core programming language used for automation.
Selenium WebDriver	Automates browser interaction and web elements.
PyTest	Primary test execution and fixture framework.
Unittest	Python built-in test framework used for a separate login test.
pytest-html	Generates HTML test execution reports.
CSV	Stores external test inputs such as login data and product name.
ConfigParser	Reads configuration values from config.ini.
Git / GitHub	Version control and project submission.
Google Chrome	Browser used for executing Selenium tests.

6. Framework Architecture

The framework follows a layered structure in which test cases use Page Object classes, while common operations are delegated to reusable utilities.

High-level flow:

Test Case → Page Object → Selenium WebDriver → Automation Exercise

Supporting components: Configuration Management, CSV Test Data, Driver Factory, Screenshot Utility, PyTest Fixture, and HTML Reporting.

Conceptual architecture:

Layer	Component	Responsibility
Test Layer	PyTest / Unittest	Defines and executes test scenarios.
Page Layer	LoginPage / ProductsPage	Contains locators and page actions.
Utility Layer	ConfigReader / CSVReader / DriverFactory / ScreenshotUtility	Provides reusable framework operations.
Configuration	config.ini	Stores application and execution settings.
Test Data	CSV	Stores external test inputs.
Reporting	pytest-html	Produces execution results in HTML format.

7. Project Structure

```
Capstone Project/
├── config/
│   └── config.ini
├── test_data/
│   └── test_data.example.csv
├── pages/
│   ├── login_page.py
│   └── products_page.py
├── tests/
│   ├── test_login.py
│   ├── test_product_search.py
│   └── test_unittest_login.py
├── utilities/
│   ├── config_reader.py
│   ├── csv_reader.py
│   ├── driver_factory.py
│   └── screenshot.py
├── screenshots/
│   └── test_screenshot_on_failure.png
├── reports/
│   └── report.html
├── conftest.py
├── pytest.ini
├── requirements.txt
├── .gitignore
└── README.md
```

Folder descriptions:

Folder / File	Description
config/	Contains application configuration.
test_data/	Contains CSV test data template.
pages/	Contains Page Object Model classes.
tests/	Contains PyTest and Unittest test cases.
utilities/	Contains reusable framework utility classes.
screenshots/	Stores screenshots captured during failure testing.
reports/	Stores the generated HTML test report.
conftest.py	Contains PyTest fixture and failure screenshot hook.
pytest.ini	Configures PyTest discovery and HTML reporting.
requirements.txt	Lists required Python packages.

.gitignore	Prevents local credentials, virtual environment, and cache files from being committed.
README.md	Provides project documentation and execution instructions.

8. Framework Components

8.1 Page Object Model

Page-specific locators and actions are maintained separately from test cases using the Page Object Model. LoginPage handles login operations, while ProductsPage handles product navigation and product searching. This reduces duplication and keeps test scripts focused on test behavior.

8.2 Configuration Management

Application URL, browser, and timeout values are maintained in config/config.ini. ConfigReader reads these values during execution. This avoids hardcoding important configuration values directly into test cases.

8.3 Test Data Management

CSV is used for external test data. The data structure includes username, password, and search_product. The repository contains a safe example CSV; actual local credentials are excluded from version control.

8.4 Driver Factory

DriverFactory provides a centralized method for creating and configuring the Selenium Chrome WebDriver.

8.5 Utility Classes

The utilities folder contains ConfigReader, CSVReader, DriverFactory, and ScreenshotUtility. These classes provide reusable operations and reduce duplicate code.

8.6 PyTest

PyTest is used as the primary test execution framework. The login and product search scenarios are implemented as PyTest test cases. A PyTest fixture in conf/test.py initializes the browser and opens the application before execution.

8.7 Unittest

A separate login test is implemented using Python's built-in Unittest framework in tests/test_unittest_login.py. It demonstrates setUpClass(), tearDownClass(), and Unittest assertions.

8.8 Screenshot on Failure

The PyTest fixture checks whether the test call phase failed. If it did, ScreenshotUtility captures the browser screen and saves it in the screenshots folder.

8.9 HTML Reporting

pytest-html is configured through pytest.ini to generate a self-contained HTML execution report at reports/report.html.

9. Test Scenarios

9.1 Login

1. Launch Chrome browser.
2. Open Automation Exercise.
3. Navigate to the Login page.
4. Read username and password from CSV test data.
5. Enter the email address.
6. Enter the password.
7. Click Login.
8. Verify that the logged-in state is displayed.

9.2 Product Search

9. Launch Chrome browser.
10. Open Automation Exercise.
11. Navigate to the Products page.
12. Read the product name from CSV test data.
13. Enter the product name in the search field.
14. Click the Search button.
15. Verify that the searched product is displayed.

10. Implementation Details

10.1 Login Page

LoginPage contains the locators and methods needed for login. The implementation uses Selenium's By strategies and explicit waits through WebDriverWait and expected_conditions.

10.2 Products Page

ProductsPage contains navigation, product search, and product verification methods. The search result is validated using a product name locator.

10.3 CSV Reader

CSVReader uses Python's csv module and DictReader to load the CSV rows as dictionaries. This allows tests to retrieve values by column name.

10.4 PyTest Fixture and Failure Handling

The driver fixture creates the browser using DriverFactory, opens the configured base URL, yields control to the test, and then performs failure screenshot handling before closing the browser.

10.5 Test Configuration

pytest.ini configures test discovery and automatically generates a self-contained HTML report when pytest is executed.

11. Test Execution

The framework can be executed using the following commands:

```
python -m venv venv
```

```
venv\Scripts\activate
```

```
pip install -r requirements.txt
```

```
pytest
```

For local execution, a test_data.csv file containing valid test credentials must be created from the provided example template. Actual credentials should not be committed to GitHub.

12. Test Results and Outputs

Test Case	Framework	Result
test_valid_login	PyTest	PASSED
test_search_product	PyTest	PASSED
test_valid_login_unittest	Unittest	PASSED

Metric	Result
Total Tests	3
Passed	3
Failed	0
Skipped	0

The complete framework execution completed successfully with all three implemented tests passing.

13. Screenshots and Evidence

- PyTest terminal output showing 3 passed:

```
PS C:\Users\SUTANU\Desktop\Wipro-Sutanu-Pal> .\venv\Scripts\Activate.ps1
(venv) PS C:\Users\SUTANU\Desktop\Wipro-Sutanu-Pal> pytest
===== test session starts =====
platform win32 -- Python 3.13.1, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\SUTANU\Desktop\Wipro-Sutanu-Pal
plugins: html-4.2.0, metadata-3.1.1
collected 3 items

Folder 2 - Capstone Project\tests\test_login.py . [ 33%]
Folder 2 - Capstone Project\tests\test_product_search.py . [ 66%]
Folder 2 - Capstone Project\tests\test_unittest_login.py . [100%]

===== 3 passed in 66.41s (0:01:06) =====
(venv) PS C:\Users\SUTANU\Desktop\Wipro-Sutanu-Pal> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& c:\Users\SUTANU\Desktop\Wipro-S
utanu-Pal\venv\Scripts\Activate.ps1)
(venv) PS C:\Users\SUTANU\Desktop\Wipro-Sutanu-Pal>
```

- HTML report showing 3 passed, 0 failed, and 0 skipped:

report.html

Report generated on 08-Sep-2026 at 12:46:52 by `pytest-html v4.2.0`

Environment

Python	3.13.1
Platform	Windows-11-10.0.26200-SP0
Packages	- pytest: 9.1.1 - pluggy: 1.6.0
Plugins	- html: 4.2.0 - metadata: 3.1.1

Summary

3 tests took 00:01:12.

(Un)check the boxes to filter the results.

☐ 0 Failed, ☒ 3 Passed, ☐ 0 Skipped, ☐ 0 Expected failures, ☐ 0 Unexpected passes, ☐ 0 Errors, ☐ 0 Reruns, ☐ 0 Retried,

[Show all details](#) / [Hide all details](#)

Result	Test	Duration	Links
Passed	tests/test_login.py::TestLogin::test_valid_login	00:00:25	
Passed	tests/test_product_search.py::TestProductSearch::test_search_product	00:00:23	
Passed	tests/test_unittest_login.py::TestLoginUnit::test_valid_login_unittest	00:00:23	

- GitHub Folder 2 showing the complete project structure:

Name	Last commit message	Last commit date
..		
config	Add Selenium Python automation framework	4 days ago
pages	Add Selenium Python automation framework	4 days ago
reports	Add project requirements and update test report	4 days ago
screenshots	Add Selenium Python automation framework	4 days ago
test_data	Add Selenium Python automation framework	4 days ago
tests	Add Selenium Python automation framework	4 days ago
utilities	Add Selenium Python automation framework	4 days ago
.gitignore	Add Selenium Python automation framework	4 days ago
README.md	Update framework components in README	4 days ago
conftest.py	Add Selenium Python automation framework	4 days ago
pytest.ini	Add Selenium Python automation framework	4 days ago
requirements.txt	Add project requirements and update test report	4 days ago

14. Challenges and Solutions

Challenge	Solution
Keeping test scripts maintainable.	Used Page Object Model to separate page actions from test cases.
Avoiding hardcoded configuration.	Moved URL, browser, and timeout settings into config.ini.
Separating test data from test code.	Used CSV test data and a reusable CSVReader.
Handling dynamic web elements.	Used explicit waits with WebDriverWait and expected conditions.
Debugging failed PyTest executions.	Implemented automatic screenshot capture on PyTest failure.
Providing execution evidence.	Configured pytest-html to generate a self-contained HTML report.
Protecting credentials in GitHub.	Excluded the real local test_data.csv through .gitignore and committed only an example template.

15. Conclusion

The project successfully demonstrates the development of a reusable Selenium Python automation framework for an E-Commerce web application. The framework implements the required automation scenarios of Login and Product Search and incorporates Page Object Model, PyTest, Unittest, utility classes, configuration management, CSV test data, screenshot capture on failure, and HTML reporting.

The final execution produced three passing tests with no failures or skipped tests. The project has also been organized and documented in GitHub under the designated Capstone Project folder.

16. Future Enhancements

- Add more E-Commerce scenarios such as registration, cart, checkout, and logout.
- Add support for additional browsers such as Firefox and Edge.
- Introduce a reusable BasePage class for common Selenium operations.
- Improve HTML report integration by attaching failure screenshots directly to reports.
- Add logging for detailed execution diagnostics.
- Integrate the framework with a CI/CD platform such as Jenkins or GitHub Actions.
- Expand data-driven testing with multiple CSV records.

17. References

1. Selenium WebDriver documentation: <https://www.selenium.dev/documentation/>
2. PyTest documentation: <https://docs.pytest.org/>
3. Python Unittest documentation: <https://docs.python.org/3/library/unittest.html>
4. Automation Exercise: <https://automationexercise.com/>
5. pytest-html documentation: <https://pytest-html.readthedocs.io/>

18. GitHub Repository

The complete source code, README, test outputs, screenshots, and HTML report are maintained in the project's GitHub repository.

GitHub Repository:

<https://github.com/sutanu-pal/Wipro-Sutanu-Pal>

Capstone Project Location:

Folder 2 - Capstone Project

.